Developer Role

Week 2 Role Research | Prepared by Trinh Hieu Tran

Role summary	A developer designs, builds, tests, and maintains the software features of an application. The role focuses on turning requirements into working code, connecting the user interface to backend services, and making sure the application behaves correctly for users.

Main responsibilities

- Understand user requirements and convert them into practical application features.
- Build frontend screens, backend APIs, validation logic, and integration between system layers.
- Write clean, readable, and maintainable code using the selected project architecture.
- Use GitHub for version control, commits, collaboration, and evidence of weekly progress.
- Fix bugs, improve usability, and support testing by explaining how features should work.

Key tools and skills

- React for user interface development and reusable frontend components.
- ASP.NET Core for backend API development and controller logic.
- C# for server-side programming and application rules.
- Git and GitHub for source control and collaboration.
- VS Code or Visual Studio for coding, debugging, and project setup.

Database Administrator Role

Week 2 Role Research | Prepared by Trinh Hieu Tran

Role summary	A Database Administrator plans, organises, protects, and maintains the data layer of a system. The role focuses on database structure, data accuracy, backups, access control, performance, and making sure application data can be stored and retrieved reliably.

Main responsibilities

- Design database tables, relationships, keys, and constraints based on system requirements.
- Maintain data quality by defining required fields, valid values, unique records, and date rules.
- Prepare database scripts, migrations, seed data, and backup or restore procedures.
- Support developers by ensuring the database matches the application models and API needs.
- Monitor privacy and security concerns so user records and application details are handled responsibly.

Key tools and skills

- PostgreSQL for relational database storage.
- Entity Framework Core for models, migrations, and database access from .NET.
- SQL for querying, filtering, reporting, and checking data quality.
- Docker for running a local database environment consistently.
- Database design concepts such as primary keys, foreign keys, indexes, and normalization.